

Bachelor Thesis

Comparing Machine Learning Approaches for Trade Intent Classification from Noisy Livestream Transcripts

Andreas Oberdörfer

Date of Birth: 24 December 2002
Student ID: h12217505

Subject Area: Information Business

Studienkennzahl: J 033 561

Supervisor: Johann Mitlöhner

Date of Submission: 26 July 2026

Department of Information Systems & Operations Management, Vienna University of Economics and Business, Welthandelsplatz 1, 1020 Vienna, Austria

Contents

1	Introduction	7
1.1	Motivation	7
1.2	Research Question	8
1.3	Scope	8
1.4	Structure of This Work	9
2	Related Work	10
2.1	Text Classification	10
2.2	Feature Extraction: TF-IDF	10
2.3	Classical Classification Algorithms	11
2.4	Word Embeddings and Pretrained Representations	13
2.5	Handling Class Imbalance	13
2.6	Transformer-Based Models	14
2.7	Automatic Speech Recognition	16
2.8	NLP Applications in Financial Markets	17
3	Methodology	18
3.1	System Pipeline Overview	18
3.2	Task Definition and Label Set	20
3.3	Text Normalization	21
3.4	Dataset Construction	22
3.5	Dataset Characteristics	23
3.6	Data Cleanup	24
3.7	Model Architectures	25
3.7.1	TF-IDF Feature Extraction (Shared by Classical Models)	26
3.7.2	Logistic Regression	26
3.7.3	Support Vector Machine	27
3.7.4	Multi-Layer Perceptron	27
3.7.5	DistilBERT (Frozen Encoder + Trained Head)	27
3.7.6	ModernBERT (Frozen Encoder + Trained Head)	28
3.7.7	Model Comparison Summary	28
3.8	Evaluation Protocol	29
3.8.1	Cross-Validation with Transcript-Level Splits	29
3.8.2	Metrics	30
3.8.3	Benchmark Artifacts	31
4	Results	32
4.1	Primary Results on the Cleaned Dataset	32
4.2	Original Dataset Baseline and Cleanup Effects	33

4.3	Per-Label Performance	34
4.4	Transformer Model Performance	35
4.5	Confusion Matrix Analysis	36
4.6	Analysing Feature Importance	38
4.7	Statistical Significance	39
4.8	MLP Results under Balanced Class Weighting	39
4.9	Embedding Visualization	40
4.10	Error Analysis	41
4.11	How Large Does the Vocabulary Need to Be?	42
4.12	From the Benchmark to Complete Sessions	43
4.13	The Full Pipeline on the Same Sessions	44
5	Discussion	46
5.1	Why the Classical Models Win Here	46
5.2	The Precision-Recall Trade-off	47
5.3	Which Actions Are Hard, and Why	47
5.4	Implications for Production Deployment	48
5.5	Ethical, Legal, and Operational Risk	48
5.6	Limitations	49
5.7	Future Work	50
6	Summary	51

List of Figures

1	The four-stage system pipeline at a glance. Highlighted is the classification stage, the component this thesis evaluates. Audio captured from a browser tab is transcribed by Whisper, sorted into trading actions, and checked for validity before any execution.	20
2	From text to prediction with a frozen encoder. Only the small classification head on the right is trained on the labelled data; the encoder stays exactly as pretrained.	28
3	Confusion matrices on the cleaned dataset for logistic regression (left) and ModernBERT (right), summed over all five folds. Each row is a true label and each column a predicted one, so correct predictions fall on the diagonal; the darker a cell, the more examples it holds. NO = NO_ACTION, EL = ENTER_LONG, ES = ENTER_SHORT, TM = TRIM, EA = EXIT_ALL, MS = MOVE_STOP, MB = MOVE_TO_BREAKEVEN.	37
4	Top-5 TF-IDF features by logistic regression coefficient for three action classes. Each subplot shows the five most predictive words or bigrams for one class. A single token (“out”) dominates EXIT_ALL with weight +5.61, while “stop” dominates MOVE_STOP at +4.62. All subplots share the same x-axis scale, making the relative dominance of top features directly comparable across classes.	38
5	t-SNE projections of frozen DistilBERT (left) and ModernBERT (right) embeddings for all 1377 cleaned examples, coloured by true label.	40

List of Tables

1	Definitions of the labels used in the trade-intent classification task.	20
2	How the labels are distributed across the original dataset (1466 file entries; 1407 unique examples once content duplicates are removed, Section 3.6).	24
3	The four dataset sizes used in this thesis: the raw and the cleaned dataset, each listed before and after exact duplicates are collapsed when the data loads. The cleanup itself removes 24 flawed examples.	25
4	Summary of model architectures and key properties.	29
5	Per-label support: the number of test examples per class on the cleaned dataset, summed across all five folds.	32
6	Five-fold cross-validation results on the cleaned dataset (primary evaluation). Bold indicates best per column.	33
7	Cleanup-only performance change (cleaned minus original). Positive values indicate improvement. The MLP row uses the unbalanced MLP configuration on both datasets, so the cleanup effect is not mixed with the later balancing change.	34
8	Per-label precision (P), recall (R), and F1 for the two TF-IDF linear models on the cleaned dataset. Supp. = support: how many test examples truly belong to the class in question.	35
9	Per-label precision (P), recall (R), and F1 for the two frozen transformer models on the cleaned dataset. Supp. = support.	36
10	Top 10 misclassification patterns for logistic regression on the cleaned dataset.	41
11	The production interpretation pipeline on the five held-out sessions (15.1 hours), compared with the standalone classifier of Section 4.12. Fires are action intents on lines that carry no action label.	45

Abstract

Day traders often stream live on platforms like YouTube, talking through their trades as they happen. Buried in that constant talk are the moments that matter: when they buy, sell, or move a stop-loss. Pulling out these signals automatically means first turning the speech into text with automatic speech recognition (ASR), then sorting each line into a trading action, a task made hard by recognition errors, casual language, and far more chatter than actual trade calls.

The thesis frames this as a seven-class problem, assigning each transcript line one trading action, and compares five machine learning methods on it. Three are classical, reading simple word-count (TF-IDF) features: logistic regression, a support vector machine, and a compact neural network. The other two are recent transformer models, DistilBERT and ModernBERT, used in a lightweight way, with their inner layers frozen so that only a small classifier on top is trained. All five face the same labelled benchmark under an identical procedure, so the comparison is fair.

The simple word-count methods come out ahead. Logistic regression is strongest, with a macro-averaged F1 of 0.70 and an F1 of 0.89 for telling action lines from commentary. The reason is that a handful of trading keywords carries most of the signal, which TF-IDF captures directly, while the frozen transformers, never tuned for the task, read this narrow vocabulary no better despite their far greater size. Keyword-driven actions such as trimming, exiting, or moving a stop are easy to flag; entries are hard, because traders phrase them in countless ways. On this small, specialised, repetitive dataset, word-count features therefore stay highly competitive, while the added complexity of transformer feature extraction does not pay off when the encoder remains frozen.

A closing test on five complete sessions held out of the model’s training then exposes what the benchmark hides: set loose on a real stream, even the best model raises far more false alarms than real signals, so on its own it is unusable. The production system therefore puts the model in a supporting role behind strict hand-written rules. Replayed over those sessions, the combined system fires about six times an hour instead of 225, and roughly one fire in six hits a real action. It still misses most labelled actions, so it is a workable alerting aid rather than a fully automatic trade copier.

Keywords: text classification, trade intent detection, TF-IDF, support vector machine, logistic regression, transformer models, DistilBERT, ModernBERT, automatic speech recognition, livestream transcription

1 Introduction

1.1 Motivation

Live-streaming has created a new form of real-time financial commentary. Day traders on YouTube and Twitch share their screens and narrate their trades as they happen, announcing in ordinary spoken language when they buy, when they sell, and when they adjust a position or a stop-loss. Anyone who wants to follow these trades faces one obvious problem: the valuable signals sit buried in hours of talk that also contains market analysis, chat replies, personal anecdotes, and long silences. Catching the moment a trader commits therefore takes unbroken attention, and a signal missed is a signal gone. Because timing separates a winning day trade from a losing one, a signal caught too late is worth little.

These streams are popular. While the markets are open, the better-known day-trading channels draw thousands of simultaneous viewers, many of them following a single instrument such as the E-mini Nasdaq futures (NQ). Unlike the written trade alerts that circulate on social media, however, spoken words are fleeting and unstructured. The line between thinking out loud (“I’m watching this level”) and a real commitment (“I’m long here”) is thin, and it depends on context.

This is the motivation for an automated system with three jobs: (1) turn the live audio into text as it happens, (2) decide which of those text lines genuinely carry a trade signal, and (3) sort each signal into a concrete action, such as “enter long,” “trim position,” or “move stop.” Recent large models like Whisper [28] have transformed that first step, converting speech to text at an accuracy close to that of a human transcriber.

Steps two and three are a text classification problem: given a short transcript line and the current trading context, pick one label from a fixed list. Text classification is one of the most heavily studied problems in natural language processing (NLP) [15]. Classical methods, which count words and pass those counts to a simple linear model, have long worked well on tasks such as separating positive reviews from negative ones, or spam from legitimate email [12, 15]. Large pretrained models called transformers, among them BERT [6], later set new records across many NLP benchmarks by learning rich, context-aware representations of words from enormous amounts of text. A richer model does not automatically win when the labelled sample is small, though: survey evidence indicates that deep-learning methods need more data than traditional alternatives and may not beat them on small text datasets [15]. Hence the central question of this thesis: on a task like this one, is a transformer’s added complexity worth its cost?

The domain confronts a classifier with several problems at once. The language is casual and full of trader slang (“peel some off,” “add to the runner,” “move to breakeven”). The speech-to-text stage adds errors of its own, because specialist terms and sound-alike words are often misheard, which changes how the text looks to a classifier; Section 2.7 gives concrete examples. The classes are also severely imbalanced, since most of what a trader says is commentary rather than a real trade call. A fourth problem is that one action can be phrased in countless ways: a trader going long (that is, buying) might say “I’m long here,” “let’s try it,” “small size,” or simply “here we go.” Finally, the dataset assembled for this work is small by today’s NLP standards, with fewer than 1500 labelled examples. Together these traits make the task a good test case for comparing classical and modern model families under realistic conditions.

1.2 Research Question

This work sets out to answer one central question:

On noisy livestream transcripts, which route classifies trade intent most accurately: classical TF-IDF methods, or pretrained transformer models? And when the dataset is small and specialised, does a transformer’s added complexity actually buy any measurable gain over the simpler methods?

Answering it means putting five models, three classical and two transformer-based, on equal footing: the same labelled data for training and testing, and the same scoring procedure. Any difference in the results then comes from the model rather than from the setup. Two further questions accompany the main one. First, which trading actions are easy to detect automatically, which are hard, and why? Second, when a model misreads a line, what kinds of errors recur, and how do they relate to the way traders talk?

1.3 Scope

The scope of this thesis is deliberately narrow. The classification task is studied on its own: each model receives a short, ready-made piece of text containing one transcript line plus its trading context, and returns a single predicted action. The speech-recognition step before it and the trade-execution step after it are described for context but are not tested or tuned here. Only English-language streams are used. All labelled data comes from recorded

streams of a single YouTube streamer, FlowzoneTrader, who day-trades the E-mini Nasdaq futures (NQ), so the results may not carry over to other traders, instruments, or styles.

All transformer models are also used with their encoder frozen: the large pretrained part of the model that reads the text stays unchanged, and only a small classifier on top is trained on the labelled data. Retraining the transformers in full (called fine-tuning) lies outside the controlled comparison, because it would require more compute and extra tuning choices and would risk overfitting this small dataset. The same applies to LoRA [10], a lighter form of fine-tuning that adjusts only small add-on weights inside the model; it is discussed as future work (Section 5.7).

Why not simply hand the classification to a large language model (LLM), the technology behind tools such as ChatGPT? Such models could certainly do a task of this kind, and this work does use them, but only in supporting roles: they pre-label the training data (Section 3.4), and the production system can optionally ask one to confirm an ambiguous entry signal (Section 3.1). For the classification step itself they are impractical here. A live session produces a new line every few seconds for hours on end, and each line needs an immediate decision. The strongest LLMs run in the cloud, so every line would cost an API call and a network round trip, and the trading loop would depend on an outside service that can slow down, change, or fail in the middle of a session. The models compared in this thesis run locally, decide in milliseconds, and cost nothing per line.

1.4 Structure of This Work

The remainder of this thesis is organized as follows. Section 2 introduces the concepts this work builds on, from text classification with TF-IDF features to transformer models and automatic speech recognition, and reviews related research on NLP in financial markets. Section 3 describes the experiment: the system around the classifier, the exact classification task and its labels, how the dataset was built, checked, and cleaned, the five models, and the evaluation procedure. Section 4 presents the results and closes with an experiment that replays five complete held-out sessions through the best model. That experiment exposes the second central finding of this work: the benchmark accuracy does not hold up on a complete, unfiltered stream. This gap is what forces the layered production design, in which hand-written rules make the decisions and the classifier only assists. Section 5 discusses what the results mean and where their limits lie, and Section 6 summarizes the question, the procedure, and the findings.

2 Related Work

This section surveys the methods and research areas the thesis builds on. It moves from general ideas in text classification to the particular model families under evaluation, then to the speech recognition technology that produces the input data, and finally to how NLP is applied in financial markets.

2.1 Text Classification

Text classification, the assignment of predefined labels to natural-language documents, is a foundational NLP task. It covers spam filtering, sentiment analysis, topic categorization [15], and intent detection. Approaches differ mainly in two respects: how the text is represented (engineered features versus learned representations) and how complex the classifier is (linear versus nonlinear).

In their early opinion-mining work, Hu and Liu [11] extracted product features and the opinion words attached to them from unstructured reviews, using association mining and lexical resources. Their results show that targeted lexical information can be useful even without a deep language model. Later surveys take a longer view. Percha’s clinical-text-mining review [24], for instance, traces the shift from rule-based systems through machine learning to neural methods, and identifies the barriers between research advances and practical deployment. Kowsari et al. [15] likewise stress that the choice of representation and classifier depends on the application and its data, and that deep-learning approaches usually require substantially more data than traditional methods. This evidence anticipates the central tension of the present work: do pretrained transformers overcome the handicap of a small, noisy, domain-specific corpus, or does a simpler representation fit the available data better?

2.2 Feature Extraction: TF-IDF

Term Frequency-Inverse Document Frequency (TF-IDF) is a classical text representation scheme that assigns a numerical weight to each word in a document based on two factors: how frequently the term appears in the document (term frequency) and how rare it is across the entire corpus (inverse document frequency) [18, 30]. Formally, for a term t in a document d within a corpus D :

$$\text{TF-IDF}(t, d, D) = \text{tf}(t, d) \times \log\left(\frac{|D|}{|\{d' \in D : t \in d'\}|}\right) \quad (1)$$

where $\text{tf}(t, d)$ denotes the frequency of term t in document d , and the logarithmic factor downweights terms that appear in many documents. Sublinear scaling of the term frequency component ($1 + \log(\text{tf})$ instead of raw tf) is commonly applied, so that a term occurring ten times counts only a little more than one occurring twice [18].

The word *feature* appears throughout this thesis, so it is worth pinning down early. A feature is one measurable property of the input that a model can use for its decision. In TF-IDF, every distinct word seen in the training texts becomes one feature, and its value for a given document is the weighted count from Equation 1. Pairs of neighbouring words are usually included as features as well: a single word is called a *unigram*, a two-word sequence a *bigram*. The line “taking a partial here” therefore switches on the unigram features “taking,” “partial,” and “here” and the bigram features “taking partial” and “partial here” (very short words such as “a” are skipped). Each document becomes a long list of numbers, one per feature, almost all of them zero. Despite its simplicity, this representation works remarkably well for text classification, especially when paired with linear classifiers [12]. It stays interpretable, because each feature is a human-readable word or phrase; it is cheap to compute; and it picks up domain-specific vocabulary straight from the training data, without pretrained models or externally supplied word lists.

TF-IDF has one major weakness: it treats every word in isolation. Word order is ignored and so is meaning, so two lines that say the same thing in different words share no features at all. Take “I’m long here” and “just bought a few.” Both signal a buy, but because they have no words in common, TF-IDF reads them as completely unrelated. Closing exactly this gap is the job of word embeddings, covered in the next sections. Keying on exact words nevertheless suits trading talk quite well, because traders keep reaching for the same small set of action words (“trim,” “stop,” “long”); the feature-importance analysis in Section 4.6 shows this directly.

2.3 Classical Classification Algorithms

Three classical classification algorithms are evaluated in this work, all applied to TF-IDF feature vectors.

Logistic Regression is a linear model that predicts class probabilities by applying the softmax function to a linear combination of input features [9]. The softmax function turns a set of raw per-class scores into probabilities that are all positive and sum to one. For a feature vector $\mathbf{x}$ and K classes, the probability of class k is:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x} + b_j)} \quad (2)$$

The weights $\mathbf{w}_k$ and b_k are the model’s *parameters*: the adjustable numbers inside a model that training tunes. When this thesis says a model has 66 million parameters, it means 66 million such numbers. Training looks for parameter values that minimize the *cross-entropy loss*, a score measuring how wrong the predicted probabilities are; it stays small when the model assigns a high probability to the correct class, and climbs steeply when the model is confidently wrong. A penalty called L2 regularization is usually added: it grows with the size of the parameters, which discourages the model from putting extreme weight on any single feature. This is a standard defence against *overfitting*, the failure mode in which a model memorizes its training examples rather than learning patterns that carry over to new data. On small datasets like the one used here, that risk runs high. Logistic regression makes a natural baseline for text classification, thanks to its interpretability, its native probability output, and the strong record of linear models on the long, mostly zero feature vectors that TF-IDF produces [12].

Support Vector Machines (SVMs) look for the hyperplane (a flat decision boundary) that maximizes the margin in the feature space, that is, the distance separating the closest training examples of different classes [4]. When the data is linearly separable, the decision function is:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b \quad (3)$$

where $\mathbf{w}$ and b are chosen to maximize $\frac{2}{\|\mathbf{w}\|}$ subject to correct classification of all training examples. Real data can almost never be split perfectly, so a softened version is used in practice: a few training examples are permitted to sit on the wrong side of the boundary, and a setting governs how harshly such examples are punished [4]. In text classification, SVMs have a long track record of success [12]. One practical contrast with logistic regression is that an SVM yields no probabilities, just a raw score per class. Where probabilities are required, a small extra step called Platt scaling is trained afterwards, turning those raw scores into probabilities between 0 and 1 [27].

Multi-Layer Perceptrons (MLPs) are a feedforward architecture (information flows only one way, from input to output) that stacks an input layer, then one or more hidden layers whose activation functions are nonlinear, and finally an output layer [7]. For an MLP with a single hidden layer, the computation is:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_2 \cdot \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \quad (4)$$

where σ denotes a nonlinear activation function: ReLU, for instance, which simply zeroes out negative values. MLPs, unlike logistic regression and linear SVMs, can learn nonlinear decision boundaries. That can be beneficial when the classes are not linearly separable. The extra capacity comes at a cost, though: it raises the risk of overfitting, especially on small datasets.

2.4 Word Embeddings and Pretrained Representations

Word embeddings go one step further than counting words. Each word becomes a list of numbers, a vector, picked so that words close in meaning end up with close vectors. What these vectors encode is meaning, not spelling, and even simple arithmetic on them can bring relationships to the surface. In the most famous example, taking the vector for “king,” subtracting the vector for “man,” and adding the vector for “woman” lands close to the vector for “queen” [19]. Word2Vec [19, 20] learns such vectors with two related objectives: the continuous bag-of-words model predicts a target word from its surrounding words, while the skip-gram model predicts surrounding words from the target. GloVe [23] reaches a similar result from global word co-occurrence counts. Both share one limitation: every word gets a single fixed vector, no matter which sentence it appears in (these are called *static* embeddings). That breaks down for words that carry several meanings. The word “stop,” for instance, names a protective sell order in “move my stop” but is an ordinary verb in “stop talking”; a static embedding has no choice but to squeeze both meanings into the same vector.

Contextual embeddings remove this limitation by letting a word’s vector depend on the sentence around it. ELMo [25] demonstrated their value by reading each sentence in both directions and improving results on tasks such as question answering and named-entity recognition. Context-dependent representation is also central to the pretrained transformer encoders discussed in Section 2.6.

Moving from TF-IDF to static embeddings and then to contextual ones brings increasingly expressive representations. The trade-off is greater model complexity and computational cost.

2.5 Handling Class Imbalance

One further piece of background concerns the data rather than the representation. Class imbalance means that some categories hold far more training examples than others. It is a common problem in real-world classification [3].

When the data is badly skewed, a classifier can look deceptively good simply by favouring the largest category: predict the most common answer every time and the measured error stays low, while the rare categories, often the ones that actually matter, are never caught at all.

There are several remedies. *Oversampling* techniques like SMOTE [3] fabricate additional synthetic instances of the underrepresented classes; *undersampling* takes the reverse route, discarding part of the common class to even out the counts. *Cost-sensitive learning* leaves the data untouched and instead reweights the scoring, penalizing mistakes on rare classes more heavily. Logistic regression and the SVM here rely on exactly this idea through the `class_weight=balanced` setting, which hands every class a weight that rises as the class grows rarer, in inverse proportion to how often it appears. Concretely, the weight assigned to class k is:

$$w_k = \frac{n}{K \cdot n_k} \quad (5)$$

Here n is the total number of training examples, K is the number of classes, and n_k is the number of examples in class k itself. So a rare class, with few examples, gets a large weight, and each of its examples counts for more.

This thesis uses a mix of these ideas. When the training data for each experiment is assembled, the dominant `NO_ACTION` class is capped at 2.5 times the total number of action examples (Section 3.8). On top of that, the linear models and the small classifiers on top of the transformers use cost-sensitive class weighting, while the MLP, whose implementation in the scikit-learn library cannot weight classes directly, is balanced by oversampling instead (details in Section 3.7).

2.6 Transformer-Based Models

Before transformers, the dominant neural models for text were *recurrent* networks, which read a sentence strictly one word at a time, carrying a running summary along from word to word. That design has two drawbacks: the words cannot be processed in parallel, which makes training slow, and information from the start of a long passage tends to fade by the time the end is reached. The Transformer architecture, introduced by Vaswani et al. [36], removed the word-by-word reading entirely and replaced it with a *self-attention* mechanism that lets every token (a word or word piece) look directly at every other token at once. This allows parallel computation and makes long-range connections in the text just as easy to use as neighbouring words. The core self-attention operation computes:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (6)$$

where $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are query, key, and value matrices derived from the input, and d_k is the dimensionality of the key vectors. Multi-head attention extends this mechanism by running h such attention operations in parallel, each with its own learned weights, and combining their outputs. This allows the model to attend to different types of relationships simultaneously.

A complete transformer layer combines this attention step with a small feedforward network and standard stabilizing components (residual connections and layer normalization). Stacking many such layers produces increasingly abstract representations of the input. The whole stack is called the *encoder*: the part of the network whose job is to read text in and give back, for every token, a vector of numbers that captures what that token means in its particular context. Whenever this thesis speaks of “embedding” a piece of text, an encoder of this kind is doing the work.

BERT (Bidirectional Encoder Representations from Transformers) [6] used the transformer encoder to pretrain a language model on a huge amount of text. Its main training trick is fill-in-the-blank: hide a random word in a sentence and have the model guess it from the words on both sides (this is called masked language modelling). Because it sees the words before and after the blank at the same time, BERT builds a richer sense of each word than older models that read in only one direction. Once pretrained, BERT can be adapted to a specific task by adding a small output layer and training it on labelled data, a step called fine-tuning. This approach broke records across many standard NLP tests.

There are two ways to put BERT to work on a new task. The first is *fine-tuning*: start from the pretrained weights and continue training the whole model, encoder included, on the labelled task data, so that every layer adapts to the new task. This exposes all model parameters to task-specific training and therefore requires more computation and tuning choices. The second way is *feature extraction*: freeze the pretrained encoder, use it only to turn text into vectors, and train just a small classifier on top. This gives up the ability to tailor the encoder to the task, but it trains far fewer parameters. Peters et al. [26] compare these strategies directly and find that their relative performance depends on the source and target tasks rather than one strategy winning universally. This thesis uses feature extraction to keep the controlled comparison feasible under the available compute budget and to limit the number of trainable parameters on the small dataset.

DistilBERT [31] is a smaller and faster variant of BERT, produced

through a technique known as knowledge distillation [8]. Here a compact “student” model learns to imitate the behaviour of the larger “teacher” model, which in this case is BERT. The idea of compressing a large model into a smaller one in this way goes back to earlier model-compression work [2]. Crucially, the student learns from the teacher’s full set of output probabilities, not just its final answer, so it also picks up which classes the teacher considers similar. The result is about 40% smaller and 60% faster while retaining roughly 97% of BERT’s performance on the GLUE language-understanding benchmark (66 million parameters instead of 110 million).

ModernBERT [38] is a recent redesign of BERT that includes many improvements developed since the original appeared. Its base version can read much longer inputs at once (up to 8192 tokens instead of BERT’s 512), uses newer and more efficient ways of paying attention to context, and was pretrained on 2 trillion tokens of text and code. At 149 million parameters, ModernBERT-base is larger than the original BERT-base model (110 million). The ModernBERT paper nevertheless reports high speed and memory efficiency as well as state-of-the-art results across a broad evaluation suite. The technical labels behind these gains include rotary position embeddings [33] and faster attention kernels [5]. For the present comparison, the important distinctions are that ModernBERT is newer, larger, and designed for substantially longer inputs than DistilBERT.

These models shine on big standard benchmarks. How well they do on small, specialised datasets, especially when full fine-tuning is not an option, is still an open question. This thesis adds evidence by testing frozen transformer encoders as feature extractors on a new classification task.

2.7 Automatic Speech Recognition

The input data for the classification task in this work comes from automatic speech recognition (ASR). In recent years the field of ASR has undergone a paradigm shift, moving from pipeline-based systems (acoustic model, language model, decoder) to end-to-end neural models that map audio waveforms directly to text.

Whisper [28] is OpenAI’s large speech-to-text model, trained on 680000 hours of audio gathered from the internet. Two stages do the work: one part listens to the audio and converts it into an internal numerical form, while the second reads that form and produces the text word by word. Its training data is “weakly supervised,” meaning that the paired audio and transcripts were collected from the web rather than produced as a uniformly curated transcription corpus. Those pairs can contain noisy labels. The authors report that training on this large and diverse collection nevertheless yields

strong zero-shot transfer across datasets and domains.

Whisper generalizes unusually well to audio it was never specifically trained for, such as varied accents, recording quality, and subject matter. It often matches systems tuned for a single setting, a property known as *zero-shot* robustness, and on several benchmarks it comes close to human transcribers [28]. The model ships in several sizes, ranging from a 39-million-parameter version up to a 1.5-billion-parameter one, trading speed for accuracy.

Whisper is not perfect, though. Like any speech-to-text system, it stumbles on specialist jargon, fast or informal speech, overlapping voices, and poor audio. The kinds of mistakes that matter here are easiest to see in an example. Picture a trader speaking quickly and mid-thought:

“okay, VWAP’s right here, I’m gonna peel some off and move to breakeven.”

A clean transcript would preserve every word. In practice, several things can go wrong with this single line. The technical term “VWAP” (a common price indicator), heard only rarely by the model, might surface as “the WAP” or “v-wap.” A perfectly ordinary word can also be swapped for its sound-alike: “peel” (meaning to sell part of a position) might turn into “peal.” And because the trader is rushing, filler words and half-finished phrases get merged or dropped, so “I’m gonna peel some off” may shrink to something shorter and vaguer. The transcript source also divides continuous speech into short, time-stamped segments. One sentence can therefore be cut in two, leaving “I’m” at the end of one line and “long here” at the start of the next, which makes both fragments impossible to classify on their own. To reduce this, the full system stitches neighbouring segments back together.

These errors flow straight into the classification step and act as noise that every model has to deal with. The key point for this thesis is that the text the classifier sees is not just casual and messy; it has also been changed in predictable ways by the speech-to-text step itself. So any accuracy number has to be read with that upstream noise in mind. To reduce the most common errors, a clean-up step is applied between transcription and classification, described in Section 3.3.

2.8 NLP Applications in Financial Markets

A large and still-growing body of research applies natural language processing to financial data. One early contribution, from Loughran and McDonald [17], showed that general-purpose sentiment dictionaries falter on financial text:

a word such as “liability” reads as negative in everyday language yet carries a neutral, routine meaning in a financial setting. The example illustrates why financial NLP often benefits from domain-specific lexicons and language models.

Bollen et al. [1] reported that selected mood dimensions extracted from Twitter (today’s X) improved predictions of daily Dow Jones Industrial Average movements in their experimental setting, providing an early link between textual sentiment and market behaviour. Transformer-based models have since been pointed straight at financial text. FinBERT [41], a BERT model given further pretraining on financial communications, outperformed generic BERT in the three financial sentiment-classification experiments reported by its authors. For the broader field, the surveys of Xing et al. [40] and Wankhade et al. [37] map NLP applications across finance and sentiment analysis, spanning news-based prediction through social-media sentiment.

In one important respect, the present work departs from this prior literature. Where earlier studies analyse written financial texts (news articles, regulatory filings, social media posts), the focus here is *spoken* trading commentary transcribed in real time. The input is no polished written document. It is a noisy, fragmentary transcript of spontaneous speech, and that brings challenges more characteristic of dialogue systems and spoken language understanding than of traditional financial NLP.

3 Methodology

What follows is the experimental setup. It covers the system that surrounds the classifier, the precise task together with its labels, the way the dataset was assembled and cleaned, the configuration of the models, and the evaluation procedure.

3.1 System Pipeline Overview

The classification task studied in this work is one component of a larger system that the author built to automatically detect and execute trades from live YouTube day-trading streams. The complete pipeline consists of four stages:

1. **Audio capture and voice activity detection.** Audio is captured from a browser tab via the system’s browser-based frontend (built with the React web framework) and processed by a voice activity detector (based on the WebRTC standard), which splits the continuous audio stream into speech segments and discards silence and background noise.

2. **Speech-to-text transcription.** Speech segments are transcribed with Whisper [28], run through `faster-whisper`, a reimplementation of Whisper that uses the CTranslate2 inference engine and supports optimized execution on CPUs and GPUs [34]. Each segment is transcribed twice: a fast preview pass shows the user a provisional transcript almost immediately, and a more careful final pass produces the text that the rest of the pipeline uses. Both passes run the same Whisper model; the final pass simply spends more computing time on decoding, weighing more alternative readings of the audio before settling on one. A clean-up step then corrects frequent recognition errors in trading terminology (Section 3.3).
3. **Intent classification.** Each finalized transcript segment is classified into a trading action. In the full production system, this decision is shared by three layers. A rule engine holds several hundred hand-written text patterns, known as regular expressions: search patterns that match specific wordings such as “I’m long here.” From these matches the rule engine proposes an initial label. A locally running machine learning classifier then assigns a probability to each possible action for the same text; these confidence scores are used to veto rule matches that look like false alarms, and to recover actions the rules missed. An optional third layer can send ambiguous entry signals to a cloud-based large language model for confirmation. The machine learning classifier in this stage is the component evaluated in this thesis.
4. **Risk validation and trade execution.** Classified trade signals pass through a risk engine that checks for stale signals, insufficient confidence, and position-size limits before orders are submitted to the NinjaTrader 8 trading platform via a local HTTP bridge.

The controlled model comparison focuses on stage 3: given a transcript segment and its trading context, predict the correct action label. Every other stage is treated as fixed infrastructure. The complete pipeline architecture is illustrated in Figure 1.

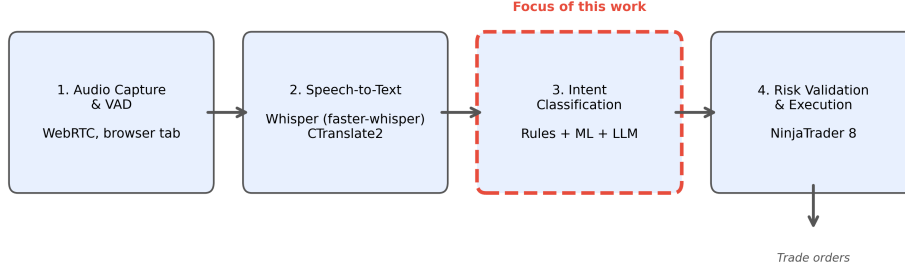


Figure 1: The four-stage system pipeline at a glance. Highlighted is the classification stage, the component this thesis evaluates. Audio captured from a browser tab is transcribed by Whisper, sorted into trading actions, and checked for validity before any execution.

3.2 Task Definition and Label Set

For every input example, the classification task assigns one of seven labels. Table 1 gives the labels and their definitions.

Table 1: Definitions of the labels used in the trade-intent classification task.

Label	Role	Description
NO_ACTION	Neutral	No trade signal detected
ENTER_LONG	Entry	Open a new long position
ENTER_SHORT	Entry	Open a new short position
TRIM	Management	Reduce position size (partial profit-taking)
EXIT_ALL	Management	Close the entire position
MOVE_STOP	Management	Adjust the stop-loss level
MOVE_TO_BREAKEVEN	Management	Move the stop-loss to the entry price

Each input example is a structured text prompt. It encodes the traded instrument, the trader’s current position state, and three overlapping windows of transcript text:

```

symbol=MNQ 03-26
position=LONG
last_side=LONG
recent=there s a nice shove up. we cleared out
analysis=there s a nice shove up. we cleared out there.
        so you got to take a partial

```

`current=there`. So, you got to take a partial

The three text windows (`recent`, `analysis`, `current`) give overlapping views of the same moment. `current` holds the transcript line being classified; `recent` holds the line spoken just before it; and `analysis` stitches those two together, so the classifier sees not only the current utterance but also what came immediately before. The `position` field holds the trader’s current state (long, short, or flat), and `last_side` records the side of the most recent entry, which stays available even after the position is closed. This state is essential for telling management actions (which require an open position) from entry actions (which open one). The `symbol` field reads MNQ rather than NQ because the copying system executes its trades on the Micro E-mini Nasdaq contract (MNQ), a smaller-sized version of the NQ contract the streamer trades.

The overlapping windows address a core difficulty of streaming transcription (Section 2.7): the archived captions divide speech into short lines, and the live system’s voice-activity detector likewise caps a segment at about four seconds. A single utterance may therefore be cut in half, and the overlap lets the classifier reconstruct its intent when the decisive words are split across two lines.

Encoding the position state as plain text inside the prompt, rather than as a separate input, is done on purpose: it lets a single text-processing pipeline handle both the words and the trading context. How each model family digests these state lines is described in Section 3.7.

3.3 Text Normalization

Between transcription and classification sits a clean-up step. It corrects the recognition errors that occur most often in this domain, so that the classifier sees consistent text. The step consists of 14 hand-written search-and-replace rules plus three general clean-ups, all applied as plain, deterministic text substitutions; no machine learning is involved.

Many of the rules simply reassemble terms that Whisper tends to break apart. Traders use contract names as shorthand, and when those names are spelled out letter by letter, “m n q” becomes “mnq,” “e s” becomes “es,” and the price benchmark, heard as “v w a p” or even as “view up,” becomes “vwap.” A second group targets sound-alike words in their trading sense, firing only when the surrounding context makes that meaning unambiguous: a trader “paying myself a peace” is corrected to “piece,” and “got my ad on” becomes “got my add on.” Tying each correction to a specific phrasing stops the rules from corrupting genuine uses of the original words. A third group

standardizes compounds that Whisper spells inconsistently, so “break even” and “broke even” both become “breakeven,” matching the spelling used in the labelled data. Three general clean-ups then run over everything. All text is lowercased, typographic apostrophes give way to plain ones, and the commas inside numbers are stripped, so that a price like “24,600” reads as the single number “24600” rather than as two separate pieces.

Simple as they are, these substitutions reduce the vocabulary fragmentation that recognition errors cause. Without them, “vwap,” “v w a p,” and “view up” would appear to the TF-IDF vectorizer and the transformer tokenizers as three unrelated expressions, even though the trader uttered the same word every time.

3.4 Dataset Construction

Construction of the training dataset proceeded in three stages: AI-assisted labelling, human review, and a merge with normalization.

Stage 1: AI labelling. Large language models processed trading livestream transcripts archived from YouTube, assigning action labels to transcript lines from the speaker’s words and trading context. Three LLMs were used across different annotation passes. Claude Sonnet produced 278 labelled examples in an initial pass, Gemini Pro added 78 examples in a validation run, and Gemini Flash produced 1139 examples in a bulk labelling pass. This first pass is related to the principle of weak supervision, in which imperfect programmatic sources produce provisional labels at scale [29]. It is not the same procedure as Snorkel: Snorkel combines heuristic labelling functions and estimates their reliability, whereas this work uses LLM proposals followed by direct human review of every proposal. The saving is substantial despite that review (Stage 2): the LLMs did the reading, working through over a hundred transcripts and condensing them into a list of proposed labels, so the reviewer only had to judge those proposals instead of combing through hours of transcript line by line. Such automated annotation makes mistakes, as any labelling method does: an LLM can mislabel sarcasm, hypothetical talk, or a half-finished sentence. The stages that follow catch many of these errors, though not all.

Stage 2: Human review. The author manually reviewed every AI-generated label, accepting, rejecting, or correcting it. Each review decision was stored in the dataset as a metadata field, which makes downstream analysis of labelling reliability possible. This human-in-the-loop arrangement pairs the scalability of automated labelling with the quality control of manual review, though review reduces errors rather than eliminating them.

Stage 3: Merging and normalization. The three reviewed collections

were merged into one dataset; where two collections had labelled the same line, a rare event, the newer label took precedence. The merge also translated two older label types into the final seven-label scheme. Lines in which the trader only mentions a possible future trade (“looking for shorts up here”) without acting became `NO_ACTION`, since such lines are commentary rather than an executed action; 437 lines were affected. A handful of lines where the trader enlarges an existing position were folded into the matching entry label, because adding contracts amounts, from an execution standpoint, to another entry in the same direction. The final dataset holds 1466 examples drawn from 103 transcripts.

Two caveats attach to every one of these labels. A label captures what the trader *announced*, not what can be verified to have happened: when a streamer says “I’m long here,” the dataset takes that entry at face value. There were no broker statements or account records to consult, so an announced trade can never be proven to have been placed. The second caveat concerns provenance. Every label passed human review, yet the labels started life as AI output, and the dataset cannot be presumed perfectly clean. The AI may have overlooked real actions altogether, and because not every transcript was re-read line by line, those missed labels would slip past unnoticed. Section 5.6 returns to both points.

Verifying the labels by hand. To gauge how far the labels can be trusted, they were spot-checked as well. The 10 transcripts carrying the most action labels, around 200 labels in all, were re-read; for each labelled line the neighbouring lines were consulted to judge whether the label matched what the trader was actually doing at that moment. Nearly every checked label held up, and not one was clearly wrong. That same pass confirmed the missed-label problem from the opposite side. The unlabelled lines of those very transcripts hold roughly 230 likely actions that carry no label, mostly repeats of an action announced moments earlier, small profit-takes, or stop adjustments mentioned in passing. What is labelled is almost always correct; not everything that happened is labelled. Some lines tagged `NO_ACTION` may therefore harbour weak or repeated actions, and this blurs the boundary between action and non-action during both training and evaluation. The benchmark in this thesis should consequently be read as a controlled comparison of models on the labelled examples, not as a measurement of how well trade signals are detected in raw livestream speech.

3.5 Dataset Characteristics

Table 2 shows the label distribution of the original dataset.

Table 2: How the labels are distributed across the original dataset (1466 file entries; 1407 unique examples once content duplicates are removed, Section 3.6).

Label	Count	Percentage
NO_ACTION	569	38.8%
EXIT_ALL	271	18.5%
TRIM	267	18.2%
ENTER_SHORT	153	10.4%
MOVE_STOP	97	6.6%
ENTER_LONG	94	6.4%
MOVE_TO_BREAKEVEN	15	1.0%
Total	1466	100%

The distribution is heavily imbalanced, and it is worth explaining where the NO_ACTION examples come from. They are labelled lines like all the others: partly lines that the AI annotators explicitly marked as non-action commentary, and partly the 437 remapped future-trade lines described above. They are *not* a random sample of all the chatter in the streams. That is why NO_ACTION makes up only 38.8% of this dataset, even though in a complete, uninterrupted session far more than 38.8% of all spoken lines carry no action; compared to a real stream, the dataset is heavily *action-enriched*. At the other extreme, MOVE_TO_BREAKEVEN has only 15 examples (1.0%), far too few for any classifier to learn reliable patterns. Both points matter for interpreting all results; the manual check above (Section 3.4) already touched the first, and Section 5.6 returns to both.

3.6 Data Cleanup

Before model training, an automated cleanup pipeline corrected several recurring flaws in the original 1466-example dataset.

The largest fix concerns contradictory position states. Some management-label examples (for example TRIM or EXIT_ALL) carried position=FLAT, which is impossible: a trader cannot trim a position that does not exist. These contradictions survived the human review because the review judged whether the action label matched the trader’s words; the position fields were filled in automatically during labelling and were not part of that check. For 200 examples, other recorded fields (the trade side stored during labelling, or the side of the most recent trade) exposed the true position, so the state

could be repaired. The few contradictory examples that resisted repair were dropped.

Two further, smaller categories fell away in cleanup. One was duplicate announcements. Because speakers on a livestream tend to restate themselves, a single action could be labelled twice only a few lines apart (“piece off into 50s,” then “partial here, folks” two lines later; both refer to the same trim), and the cleanup kept only the earliest mention. The other category was talk aimed at the audience: a handful of lines coach the viewers instead of reporting the trader’s own position (“you must pay yourself here”). Treating those as real executions would train the classifier toward precisely the false positive the system is built to suppress. Once all of this is done, **1442 clean examples** drawn from 103 transcripts remain.

A final deduplication step runs automatically each time the data loads. Whenever an entry matches another character for character across its label, prompt text, and position fields, the two are merged into a single record. These duplicates appear because overlapping chunks of speech can yield the identical prompt twice; left in place, a line would be counted twice over during training and testing. The effect is to reduce the original dataset from 1466 to 1407 unique examples and the cleaned dataset from 1442 to 1377.

This yields four dataset sizes, which differ only by the two mechanical steps just described. Table 3 lists them side by side.

Table 3: The four dataset sizes used in this thesis: the raw and the cleaned dataset, each listed before and after exact duplicates are collapsed when the data loads. The cleanup itself removes 24 flawed examples.

Stage	Examples	Role in this thesis
Raw merged dataset	1466	Input to the cleanup pipeline
After cleanup (24 removed)	1442	Cleaned dataset on disk
Raw, duplicates collapsed	1407	Baseline (Section 4.2)
Cleaned, duplicates collapsed	1377	Primary results (Section 4.1)

All example counts and per-label support figures reported in Section 4 refer to the deduplicated totals. The cleaned, deduplicated dataset (1377 examples) is the primary basis for comparison; results on the original dataset are reported once, to measure the effect of cleanup.

3.7 Model Architectures

This section describes how each model is set up. All models were implemented in Python, using scikit-learn [22] for the classical models and the Hugging

Face Transformers library [39] with PyTorch [21] for the transformer models.

3.7.1 TF-IDF Feature Extraction (Shared by Classical Models)

All three classical models share the same text featurization step: TF-IDF vectorization with up to 20000 features, consisting of unigrams and bigrams, with sublinear term-frequency scaling so that very frequent words do not dominate.

The effect is intuitive. A segment like *“taking a partial here on this move up”* contains the bigram “taking partial,” which carries a high weight because it appears almost exclusively in TRIM examples, handing the classifier a strong signal. *“we got a nice shove up, let’s try it,”* by contrast, holds no explicit trading keyword; phrases such as “let’s try” turn up across many classes and carry low weight, which is exactly why vaguely phrased entries prove harder to classify than explicitly announced management actions.

Capping the vocabulary at 20000 features strikes a balance: enough coverage, but without runaway dimensionality. With fewer than 1500 training examples available, a much larger vocabulary would mostly add features seen in only one or two examples, which invites overfitting. Bigrams earn their place in this domain, because many trading actions announce themselves through two-word phrases (“small long,” “move stop,” “I’m out,” “peel off”); taken one word at a time, their words are too common to mark the action.

Because the prompt is structured, the trading context rides on these very same features. The vectorizer breaks tokens at characters such as “=”, so a state line like `position=LONG` yields the word pair “position long” as a bigram feature, while `position=FLAT` yields “position flat.” From signals like these a model can learn, say, that management actions go together with an open position; the later feature-importance analysis confirms that precisely these state bigrams carry substantial weight. The structured prompt thus folds a two-part problem (text plus trading state) into one text classification problem that word-count features can manage.

3.7.2 Logistic Regression

The first baseline is logistic regression in its scikit-learn implementation. It uses L2 regularization at the default strength ($C = 1$), balanced class weights, and a limit of 1000 optimization iterations. The regularization keeps any one word from controlling the outcome, and the class weights make the rare classes weigh more heavily during training. It is a simple baseline that emits probabilities on its own and handles the class imbalance through that single weighting setting.

3.7.3 Support Vector Machine

The linear SVM (`LinearSVC` in scikit-learn) uses the same feature vectors, balanced class weighting, $C = 1$, and a limit of 2000 optimization iterations. Because an SVM produces only raw scores, the Platt-scaling step described earlier is trained on top of it to turn those scores into probabilities. Scikit-learn fits this calibration through five-fold internal cross-validation on the training partition only. The SVM draws its boundary to keep the widest possible margin between the classes, which can generalize better when training examples are scarce.

3.7.4 Multi-Layer Perceptron

The MLP is a feedforward neural network with two hidden layers of 256 and 128 units, trained with the Adam optimizer [13], the algorithm that adjusts a network’s weights during training. Because scikit-learn’s `MLPClassifier` cannot weight classes directly, balance is achieved by oversampling, and the mechanics are simple: every training example is duplicated in proportion to its balanced class weight. An example of the most common class appears once, while an example of a class that is, say, eight times rarer is copied about eight times, so the network simply sees the rare classes more often. Early stopping (halting training once performance on an internal validation split stops improving) is deliberately left off: the duplicated copies would land on both sides of that internal split, making the validation portion look deceptively easy.

3.7.5 DistilBERT (Frozen Encoder + Trained Head)

The DistilBERT model (`distilbert-base-uncased`, 66 million parameters, 6 layers) is used as a frozen feature extractor [31]: the pretrained model itself is never changed and serves purely as a converter from text to numbers. The journey of one example is short, and Figure 2 illustrates it step by step. The prompt text is first split into tokens. The frozen encoder then reads all tokens together and produces, for each one, a vector of 768 numbers describing that token in its context. These per-token vectors are averaged into a single vector of 768 numbers, a compact numerical summary of the whole example. Finally, that summary goes into the classification head, the only part that is trained: it rescales the 768 numbers to a stable range and computes one score per label, and the label with the highest score is the prediction.

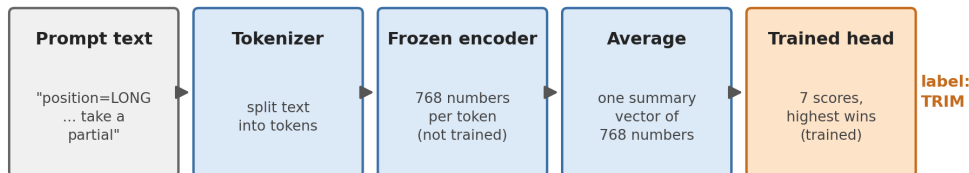


Figure 2: From text to prediction with a frozen encoder. Only the small classification head on the right is trained on the labelled data; the encoder stays exactly as pretrained.

The head is deliberately tiny: it contains only about 6900 trainable parameters, against the 66 million frozen ones. It is trained like any small neural network, using the cross-entropy loss introduced earlier to measure how wrong the predictions are, with rare classes weighted more heavily so the head cannot ignore them. The AdamW optimizer [16] adjusts the 6900 numbers during up to 20 passes through the training data.

Training only this 6900-parameter head, rather than fine-tuning all 66 million (Section 1.3), keeps the risk of overfitting low and isolates what the comparison is meant to measure: how useful each model’s pretrained representation is for this task.

3.7.6 ModernBERT (Frozen Encoder + Trained Head)

The ModernBERT model (answerdotai/ModernBERT-base, 149 million parameters, 22 layers) is used in exactly the same way as DistilBERT: frozen encoder, averaged embeddings, and the identical trained head with the identical training procedure [38]. The only difference is the encoder itself. ModernBERT has a newer design and reports pretraining on 2 trillion tokens; by comparison, the original BERT paper reports pretraining corpora containing roughly 3.3 billion words [6]. Tokens and words are not identical units, but the figures make the difference in pretraining scale clear. Whether ModernBERT’s 768-number summaries actually help on this task is precisely what the comparison measures. Despite having more than twice DistilBERT’s total parameters, ModernBERT has the same number of *trainable* parameters here, since only the head is trained. Any advantage must come from the quality of its frozen embeddings.

3.7.7 Model Comparison Summary

Table 4 summarizes the key properties of all five models.

Table 4: Summary of model architectures and key properties.

Model	Input representation	Task-trained parameters	Class-imbalance handling
Logistic regression	TF-IDF, up to 20000 features	65000 to 70000 (all)	Balanced class weights
Linear SVM	TF-IDF, up to 20000 features	0.33 to 0.35 million (five calibrated fits)	Balanced class weights
Multi-layer perceptron	TF-IDF, up to 20000 features	2.4 to 2.6 million (all)	Balanced oversampling
DistilBERT frozen encoder	Token sequence	6900 of 66 million (head only)	Weighted loss
ModernBERT frozen encoder	Token sequence	6900 of 149 million (head only)	Weighted loss

One contrast in this table is worth spelling out, because it shapes the results. The classical models train every one of their parameters on the task. The TF-IDF vocabulary varies slightly with each training fold, producing about 65000 to 70000 weights for logistic regression. Each SVM decision model has the same order of magnitude, but probability calibration stores five internally cross-validated SVM fits, so its total is about 0.33 to 0.35 million. The transformers, by contrast, keep more than 99.99% of their parameters frozen and adapt only the 6900 weights of the head. So although the transformers are vastly bigger models overall, the part that can actually specialize in trading language is about one tenth the size of the logistic-regression model and smaller still than the calibrated SVM ensemble.

3.8 Evaluation Protocol

3.8.1 Cross-Validation with Transcript-Level Splits

With fewer than 1500 labelled examples, holding out one fixed test set would be wasteful and fragile: much of the scarce data would be locked away from training, and the result would depend on which lines happen to land in the held-out set. The standard way around this is *cross-validation* [14]. The data is split into five parts, called folds. The models are then trained five times, each time on four of the parts, and tested on the remaining fifth. Every example is therefore used for testing exactly once, always by a model that never saw it during training, and the five test scores are averaged. Every model in this thesis is evaluated in exactly this way.

How the data is split into folds matters a great deal. The usual approach to cross-validation scatters individual examples at random across the folds, which works as long as those examples are independent. Here they are not. Lines drawn from one transcript carry the same speaker, the same trading day, the same recording quality, and the same recurring turns of phrase. If such lines fall into both the training and the test partition, a model can post a good score simply by recognizing the session rather than by reading the language, which inflates the reported accuracy. The name for this is data leakage. The remedy is to allocate whole transcripts to folds rather than individual lines, which guarantees that every test session is one the model has not encountered. That allocation is deterministic, driven by a fixed scrambling code (a hash) of each transcript’s file path, so the folds come out the same on every rerun. The 103 transcripts are spread across the five folds at roughly 20 apiece, yet the count of examples in each fold shifts with the length of the session and how heavily the trader traded during it (exact figures in Section 3.8.3).

Within each fold, a deterministic cap limits the number of `NO_ACTION` training examples to at most 2.5 times the total number of action examples (Section 2.5). If the cap is exceeded, the examples to keep are selected by sorting them by a hash of each example’s identity, which behaves like a random draw but produces exactly the same selection on every rerun. In this action-enriched dataset the cap is never reached, so no training examples are removed. The test part always keeps the proportions of the labelled dataset, which should not be read as the true frequency of action and non-action speech in an uninterrupted livestream.

3.8.2 Metrics

Five metrics are reported here, in line with established practice for imbalanced multi-class classification [32].

Accuracy is the fraction of examples classified correctly. It can deceive on imbalanced data: a classifier that always guesses `NO_ACTION` would reach about 39% accuracy here without catching a single trade.

Macro F1 guards against exactly that deception, and it is built from two ingredients. Precision is the share of a model’s predictions for a class that prove correct; recall is the share of that class’s true examples the model actually finds. F1 itself is the harmonic mean of precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (7)$$

Macro F1 is then the unweighted mean of the per-class F1 scores, giving

each of the seven classes equal say. A model that neglects the rare classes is visibly penalized by this metric, which makes it a fairer assessment than plain accuracy.

Action F1, the primary metric, boils the problem down to a single binary question: did the system recognize that *some* trade action occurred? An action prediction on an action example counts as a true positive; an action prediction on a `NO_ACTION` example is a false positive; a `NO_ACTION` prediction on an action example is a false negative. This gauges directly what the system is built to do, namely separate actionable signals from commentary.

Action precision is the fraction of all action predictions that were truly actions; high precision means few false alarms. **Action recall** is the fraction of all true actions that were predicted as some action; high recall means few missed signals.

One more term appears throughout the result tables: the *support* of a class is simply the number of test examples whose true label is that class. It says how much evidence stands behind each per-class score; a score computed over 13 examples (as for `MOVE_TO_BREAKEVEN`) is far less stable than one computed over 548 (as for `NO_ACTION`).

3.8.3 Benchmark Artifacts

All results in this thesis are read from stored result files written by the benchmark scripts, and the repository contains everything needed to regenerate them: the code, the labelled dataset, and the 103 recorded transcripts from which the dataset was built, among them the five held-out sessions of the full-stream experiments. The tracked dataset and transcript files together occupy about 22 MB, so they fit directly in the repository rather than requiring an external archive. An outside reader can therefore inspect every piece of code, every stored result, and the underlying data, and can rerun the experiments from the published material.

Inside the pipeline, two safeguards matter most. The TF-IDF vocabulary is learned only from the training portion of each split, never from test data. And the splits are deterministic while the classical models run with fixed random seeds (the numbers that make random choices repeatable), so reruns produce the same folds and the same scores; only the transformer heads can vary minimally between reruns, because their starting weights are drawn at random without such a seed.

Across the five folds, the split assigns 21, 21, 21, 21, and 19 transcripts (225, 339, 287, 293, and 233 test examples, respectively), for 1377 examples in total. Table 5 lists how often each label occurs among these test examples, summed over all folds; these are the support columns of the per-label tables

in Section 4.3.

Table 5: Per-label support: the number of test examples per class on the cleaned dataset, summed across all five folds.

Label	Support
NO_ACTION	548
EXIT_ALL	244
TRIM	242
ENTER_SHORT	144
MOVE_STOP	94
ENTER_LONG	92
MOVE_TO_BREAKEVEN	13

4 Results

This section presents the results in three parts. First come the main scores of all five models on the cleaned dataset, together with a comparison against the uncleaned dataset that shows what the cleanup was worth. The middle subsections then break those scores down: results per label, the typical confusions, which words drive the decisions, statistical tests, and an error analysis. Finally, the last two subsections leave the labelled dataset behind and test the system on five complete livestream sessions, first the best model on its own and then the full production pipeline.

4.1 Primary Results on the Cleaned Dataset

Table 6 presents the five-fold cross-validation results on the cleaned dataset (1377 unique examples after content deduplication, 103 transcripts). Values are reported as mean $\pm$ standard deviation across folds. For this primary evaluation, the MLP uses the balanced oversampling procedure described in Section 3.7.

Table 6: Five-fold cross-validation results on the cleaned dataset (primary evaluation). Bold indicates best per column.

Model	Accuracy	Macro F1	Action F1	Action precision	Action recall
Linear SVM	0.795 ± 0.021	0.688 ± 0.062	0.884 ± 0.011	0.900 ± 0.031	0.870 ± 0.020
Logistic regression	0.785 ± 0.021	0.703 ± 0.045	0.887 ± 0.024	0.863 ± 0.033	0.914 ± 0.024
MLP	0.763 ± 0.024	0.634 ± 0.049	0.850 ± 0.006	0.898 ± 0.037	0.809 ± 0.031
ModernBERT	0.727 ± 0.012	0.649 ± 0.060	0.872 ± 0.031	0.833 ± 0.044	0.917 ± 0.037
DistilBERT	0.676 ± 0.065	0.588 ± 0.029	0.848 ± 0.034	0.801 ± 0.070	0.909 ± 0.071

The two linear classical models achieve the strongest aggregate performance. Logistic regression reaches the best macro F1 (0.703) and action F1 (0.887), while the SVM reaches the best accuracy (0.795) and action precision (0.900). ModernBERT has the highest action recall (0.917), but this comes with lower precision and lower macro F1. The MLP, after balanced oversampling, occupies a middle position: it is more conservative than the transformers and less accurate than the two linear models.

4.2 Original Dataset Baseline and Cleanup Effects

To quantify the impact of data cleanup, every model was also evaluated on the original dataset (1407 unique examples after content deduplication, 103 transcripts) using the same protocol. These original-dataset scores serve only as a secondary baseline and are not used for the per-label and error analyses; instead of a second full table, the effect of cleanup is reported directly as a difference. Table 7 gives, for each model, the change in the three central metrics (cleaned minus original). For the MLP, the unbalanced configuration is used on both datasets, so the cleanup effect is not mixed with the separate balancing change.

Table 7: Cleanup-only performance change (cleaned minus original). Positive values indicate improvement. The MLP row uses the unbalanced MLP configuration on both datasets, so the cleanup effect is not mixed with the later balancing change.

Model	Δ Accuracy	Δ Macro F1	Δ Action F1
Linear SVM	+0.020	+0.018	+0.014
Logistic regression	+0.022	+0.023	+0.016
ModernBERT	+0.048	+0.047	+0.017
DistilBERT	+0.038	−0.007	+0.011
MLP (unbalanced)	−0.006	−0.019	−0.005

Most metrics rose after cleanup. The largest beneficiary was ModernBERT, with +0.048 in accuracy and +0.047 in macro F1; logistic regression moved far less, gaining only +0.022 and +0.023. DistilBERT was mixed: accuracy and action F1 climbed, but its macro F1 slipped a little. Cleanup alone did not help the unbalanced MLP; its scores even slipped slightly. That points to class-imbalance handling, rather than the particular noisy lines that cleanup stripped out, as the source of its earlier weakness. Balanced over-sampling tells a different story. Applied to the MLP on the same cleaned data (Section 4.8), it lifts accuracy from 0.726 to 0.763, macro F1 from 0.572 to 0.634, and action F1 from 0.804 to 0.850, a substantial shift. In absolute terms the cleanup gains stay small, but they confirm the familiar “garbage in, garbage out” rule: feed the same models slightly cleaner data and they learn measurably better.

4.3 Per-Label Performance

Aggregated across all five folds, Table 8 reports the per-label precision, recall, and F1 of the two linear models on the cleaned dataset.

Table 8: Per-label precision (P), recall (R), and F1 for the two TF-IDF linear models on the cleaned dataset. Supp. = support: how many test examples truly belong to the class in question.

Label	Logistic regression			Linear SVM			Supp.
	P	R	F1	P	R	F1	
NO_ACTION	0.857	0.777	0.815	0.812	0.850	0.831	548
ENTER_LONG	0.574	0.717	0.638	0.684	0.587	0.632	92
ENTER_SHORT	0.614	0.618	0.616	0.676	0.521	0.588	144
TRIM	0.786	0.835	0.810	0.797	0.860	0.827	242
EXIT_ALL	0.811	0.861	0.835	0.829	0.857	0.843	244
MOVE_STOP	0.818	0.862	0.839	0.833	0.851	0.842	94
MOVE_TO_BREAKEVEN	0.800	0.308	0.444	0.750	0.231	0.353	13

A clear difficulty hierarchy emerges. The management labels (TRIM, EXIT_ALL, MOVE_STOP) land between F1 scores of 0.810 and 0.839, a reflection of their explicit, distinctive keywords (“partial,” “taking profit,” “I’m out,” “move my stop”). The two entry labels prove considerably harder. Their F1 scores hover between 0.62 and 0.64, since entry language tends to be more varied and ambiguous (“let’s try it,” “here we go,” “small size long here”). MOVE_TO_BREAKEVEN, the rarest class, climbs to F1 = 0.444 after cleanup, yet with only 13 support examples it remains too sparse for reliable learning.

The SVM follows the same overall pattern, though it strikes a different trade-off on the entry classes. When it does call an entry, it is right more often (precision 0.684 on ENTER_LONG, against 0.574 for logistic regression) but it makes that call far less frequently (recall 0.587 versus 0.717). This makes it the more cautious of the two on precisely the classes where phrasing varies most.

4.4 Transformer Model Performance

Table 9 presents the per-label results for the two transformer models side by side.

Table 9: Per-label precision (P), recall (R), and F1 for the two frozen transformer models on the cleaned dataset. Supp. = support.

Label	DistilBERT			ModernBERT			Supp.
	P	R	F1	P	R	F1	
NO_ACTION	0.825	0.637	0.719	0.855	0.712	0.777	548
ENTER_LONG	0.441	0.565	0.495	0.516	0.533	0.524	92
ENTER_SHORT	0.436	0.542	0.483	0.497	0.611	0.548	144
TRIM	0.651	0.756	0.700	0.714	0.826	0.766	242
EXIT_ALL	0.720	0.738	0.729	0.709	0.771	0.739	244
MOVE_STOP	0.681	0.862	0.761	0.821	0.830	0.825	94
MOVE_TO_BREAKEVEN	0.429	0.231	0.300	0.667	0.462	0.546	13

Both frozen transformer feature extractors show lower precision on entry classes than the two linear models. DistilBERT achieves 0.441 precision on `ENTER_LONG` (versus 0.574 for logistic regression), while ModernBERT reaches 0.516. The transformers do achieve relatively high recall on several action classes; DistilBERT detects 86.2% of `MOVE_STOP` actions, but frequent false positives lower their macro F1.

Despite the extreme rarity of this class, ModernBERT reaches the highest F1 on `MOVE_TO_BREAKEVEN` (0.546) among all five models. Its contextual embeddings may capture similarities between “breakeven” language and other stop-management language, but with only 13 examples this result is too unstable to support a firm explanation, and the edge on this single rare label does not carry over into the best aggregate score.

Otherwise the transformers follow the same difficulty hierarchy as the linear models: management labels are classified reliably, and entry labels stay harder and swing more from one model to the next.

4.5 Confusion Matrix Analysis

A confusion matrix is the clearest way to read off a model’s error patterns. Each row collects the examples of one true label, and the columns record what the model actually predicted. A hit therefore sits on the diagonal, and each off-diagonal cell tallies one particular type of mistake. Figure 3 presents two such matrices: one for logistic regression, which leads on both macro F1 and action F1, and one for ModernBERT, the better of the two transformers. Both are summed over all five folds of the cleaned dataset.

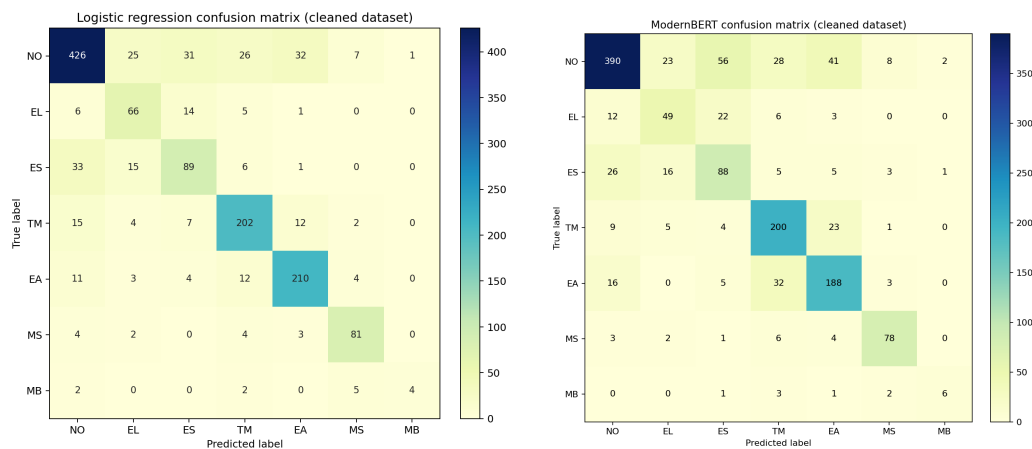


Figure 3: Confusion matrices on the cleaned dataset for logistic regression (left) and ModernBERT (right), summed over all five folds. Each row is a true label and each column a predicted one, so correct predictions fall on the diagonal; the darker a cell, the more examples it holds. NO = NO_ACTION, EL = ENTER_LONG, ES = ENTER_SHORT, TM = TRIM, EA = EXIT_ALL, MS = MOVE_STOP, MB = MOVE_TO_BREAKEVEN.

In the logistic regression matrix, several error patterns stand out.

Confusing the direction of an entry. 14 of 92 true ENTER_LONG examples are misclassified as ENTER_SHORT; conversely, 15 of 144 true ENTER_SHORT examples are misclassified as ENTER_LONG. Entry language is often structurally similar regardless of direction, differing only in a directional keyword (“long” versus “short”) that may be transcribed ambiguously or may be missing from the relevant text window.

Missed actions. Between 2 and 33 examples of each action class are misclassified as NO_ACTION. This happens when the transcript contains an action but the language is too subtle for the classifier, for example a trader who says “I’m going to take a little off” rather than using the explicit keyword “partial.”

False alarms. 122 of 548 NO_ACTION examples (22.3%) are assigned an action label. Some non-action segments contain language that superficially resembles action language, for example when the trader discusses a hypothetical trade, reviews a past action, or gives advice to viewers. Because the corpus has incomplete action coverage (Section 3.4), a small number of these apparent false positives may also be genuinely unlabelled actions. The rare MOVE_TO_BREAKEVEN class barely registers in these totals; its most frequent error lands on MOVE_STOP, which is at least the most closely related class.

The ModernBERT matrix in the right panel runs the other way. Of the

548 `NO_ACTION` examples, logistic regression gets 426 right; ModernBERT manages only 390, DistilBERT only 349. Both more readily drop a non-action line into one of the action columns. Put plainly, an unsure frozen transformer leans towards calling an action. This is why Table 6 pairs their high action recall with low precision: they catch more of the real actions, yet they also raise more false alarms. The remaining two models tilt in the opposite direction. The SVM acts like a stricter logistic regression, and the balanced MLP is the most cautious of the lot, routing 27 true `ENTER_LONG` and 50 true `ENTER_SHORT` examples into `NO_ACTION`.

4.6 Analysing Feature Importance

To see *why* the TF-IDF models do so well, the learned logistic regression weights were inspected directly. The ranking considers every feature the model uses, single words and two-word phrases alike. Figure 4 shows the five highest-weighted features for three representative action classes.



Figure 4: Top-5 TF-IDF features by logistic regression coefficient for three action classes. Each subplot shows the five most predictive words or bigrams for one class. A single token (“out”) dominates `EXIT_ALL` with weight +5.61, while “stop” dominates `MOVE_STOP` at +4.62. All subplots share the same x-axis scale, making the relative dominance of top features directly comparable across classes.

The feature weights reveal that trading action language is built from a small set of fixed, recurring phrases. `EXIT_ALL` is dominated by the single word “out” (weight +5.61); `MOVE_STOP` by “stop” (+4.62); `TRIM` by “some” (+3.51). TF-IDF preserves each of these words as a separate, directly weighted feature. The frozen transformer setup instead averages all token vectors into one dense 768-number summary, which does not expose individual keyword contributions in the same way. The high weights on

position-state bigrams such as “position flat” and “position short” also show that the model uses the state information supplied by the structured prompt (Section 3.2).

4.7 Statistical Significance

Mean scores by themselves cannot reveal whether a gap between two models is real or merely good luck with the particular folds. The standard check used here is the paired t -test. For two models scored on the same five folds, it asks whether the per-fold differences lean consistently one way or could plausibly be chance. Its p -value gives the probability of seeing differences at least this large if the models had equal mean performance; by convention, anything below 0.05 counts as evidence of a difference. Five folds leave these tests with little power, and their overlapping training sets mean that the fold scores are not fully independent. The tests are therefore exploratory checks rather than proof.

Every pair of models was run through such a test on its per-fold macro F1 scores, using the same model versions as Table 6 (for the MLP, the balanced one). These comparisons ask the natural question of any benchmark: is model A really better than model B, or only better on average? The answer is sobering. Statistically, the two linear models cannot be told apart ($p = 0.54$), and neither separates significantly from the balanced MLP. Among the classical-versus-transformer comparisons, only logistic regression against DistilBERT reaches conventional significance ($p = 0.014$); the comparisons involving ModernBERT do not. No correction for multiple comparisons is applied, so the tests are exploratory. A Bonferroni correction across all ten pairwise tests would set the threshold to 0.005, and the sole result below 0.05 would no longer count as significant. This reinforces the need for a cautious conclusion.

A look at the individual folds explains why the conclusions stay cautious. On average the linear models win, yet ModernBERT exceeds both linear models on macro F1 in one fold, and a transformer records the highest action recall in four of the five folds. The fair summary is that the linear models lead overall, but the ranking changes with the fold and the metric.

4.8 MLP Results under Balanced Class Weighting

Without oversampling, the MLP collapses on `MOVE_TO_BREAKEVEN` outright ($F1 = 0.000$) and drags down recall across the minority classes. Balanced oversampling (Section 3.7) pushes macro F1 up to 0.634 and `MOVE_TO_BREAKEVEN` F1 to 0.133; most of that improvement lands in the entry classes (`ENTER_LONG`

F1 0.533, ENTER_SHORT 0.590), while the management classes hold up well (MOVE_STOP 0.852, TRIM 0.799, EXIT_ALL 0.798). The MLP nonetheless remains behind both linear models, its macro F1 of 0.634 falling short of 0.703 and 0.688. The likely reason is capacity: with about 2.4 to 2.6 million trainable parameters against roughly 1100 training examples per fold, the added flexibility buys no better generalization in this experiment.

4.9 Embedding Visualization

If the frozen encoders organized trading language well, examples with the same label should end up with similar embedding vectors. This can be inspected visually with t-SNE [35], a technique that squashes high-dimensional vectors down to two dimensions for plotting while trying to keep similar vectors close together. If the encoder separated the classes well, points of the same colour would form visible groups in such a plot.

For each of the two transformer models, all 1377 cleaned, deduplicated examples were embedded with exactly the same procedure as in the classification experiments, so the plots show the very same vectors that the classification heads had to work with, projected down to two dimensions with t-SNE. Figure 5 shows the result.

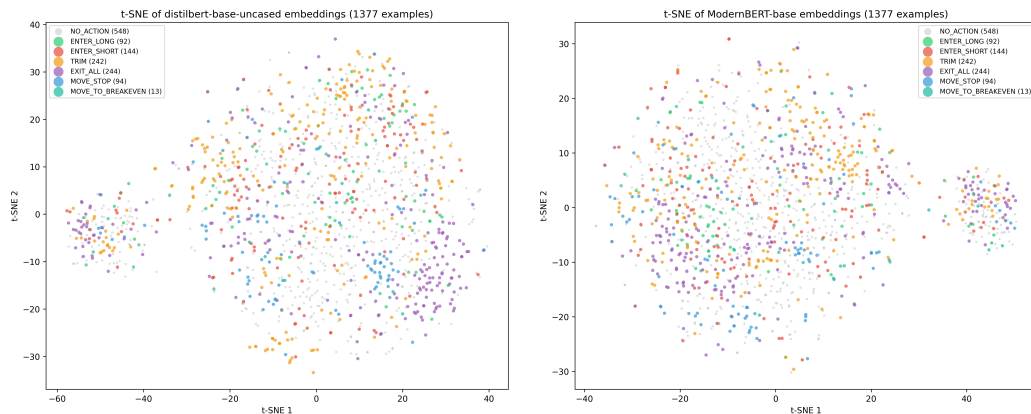


Figure 5: t-SNE projections of frozen DistilBERT (left) and ModernBERT (right) embeddings for all 1377 cleaned examples, coloured by true label.

Both plots show no clear label-level clustering. All seven classes are heavily intermixed: NO_ACTION spreads across the whole projected space, and the action classes overlap strongly with one another and with NO_ACTION. DistilBERT retains slightly more loose sub-structure, but those groupings contain mixed labels and likely reflect transcript-level or syntactic similarity

rather than trading intent, while ModernBERT’s cloud is even more uniform. In both, `MOVE_STOP` forms the most spatially coherent class, in line with how nearly every one of its examples contains the word “stop.” Because t-SNE is a visualization and not a formal test, this evidence is only illustrative; still, it agrees with the picture from the feature-importance analysis. In ordinary English, “Peeling some off” (`TRIM`) and “watching for longs” (`NO_ACTION`) can sit close together, yet the two demand different labels. TF-IDF holds them apart as distinct sparse features, whereas the frozen embeddings may place them side by side.

4.10 Error Analysis

To map the practical failure modes of the best-performing model, every misclassification from logistic regression’s five-fold cross-validation on the cleaned dataset was collected: 299 errors out of 1377 examples (21.7%). Each one comes from the test side of a fold, classified by a model that had never seen its transcript during training, so these are genuine generalization errors, not mistakes measured on training data. As noted in Section 3.4, a few of the apparent `NO_ACTION` false positives may be unlabelled actions rather than true errors. Table 10 lists the ten most frequent error patterns.

Table 10: Top 10 misclassification patterns for logistic regression on the cleaned dataset.

True Label	Predicted	Count	% of Errors
ENTER_SHORT	NO_ACTION	33	11.0
NO_ACTION	EXIT_ALL	32	10.7
NO_ACTION	ENTER_SHORT	31	10.4
NO_ACTION	TRIM	26	8.7
NO_ACTION	ENTER_LONG	25	8.4
ENTER_SHORT	ENTER_LONG	15	5.0
TRIM	NO_ACTION	15	5.0
ENTER_LONG	ENTER_SHORT	14	4.7
EXIT_ALL	TRIM	12	4.0
TRIM	EXIT_ALL	12	4.0

Looking through the misclassified examples brings out three dominant failure modes.

Indirect entry language. This is the most frequent error type (33 cases of `ENTER_SHORT` $\rightarrow$ `NO_ACTION`), and it surfaces whenever traders phrase

entries in the past tense, leave them implicit, or word them oddly. Take three examples. “I did enter short versus New York here” is past tense; “I’ve re-entered versus flow zones now” relies on an uncommon verb form; “All right. In it scalp here” carries no explicit directional keyword. Having learned to fire on standard patterns such as “short here” or “small short,” the classifier simply misses these variants.

Exit-adjacent commentary. In the second pattern (32 cases of `NO_ACTION` $\rightarrow$ `EXIT_ALL`), the model over-triggers on exit-related wording inside non-action segments. “Stopped me out by a point or two” describes a past stop-out rather than a live action; “Any push through 11s here. I’m out” is a conditional statement; “There we build out this right shoulder” is technical analysis, with “out” appearing in a non-exit context. Because the keyword “out” carries such strong weight (+5.61), the model reacts to any occurrence of it, non-action uses included.

Advisory framing. The third pattern (26 cases of `NO_ACTION` $\rightarrow$ `TRIM`) concerns second-person advice aimed at viewers that, on the surface, looks like self-directed action: “Must be taking partials here, folks,” “Cover a little bit here into 600 just in case,” and “You can cover a piece here.” Although the keywords “partials,” “cover,” and “piece” are strong `TRIM` signals, the second-person framing (“you can,” “folks”) marks these as advice, not a trade the speaker is placing. As it stands, the text normalization pipeline draws no distinction between first-person and second-person constructions.

The error patterns point to concrete improvements: the model needs to pay more attention to verb tense (did something already happen, or is it happening now?), to who is being addressed (is the trader acting, or telling viewers what to do?), and to the difference between a plan and an executed action.

4.11 How Large Does the Vocabulary Need to Be?

Here, vocabulary means the list of distinct words and word pairs that the TF-IDF step turns into features; its size was capped at 20000 in all experiments. A final check verifies that this cap is neither a hidden bottleneck nor an inflated setting. The method is an *ablation*: the same experiment is re-run several times with only one setting changed, here the maximum number of features. Logistic regression was evaluated with five-fold cross-validation on the cleaned dataset at caps of 5000, 10000, 20000, and 50000 features, yielding macro F1 of 0.700, 0.703, 0.703, and 0.703, respectively (accuracy 0.781/0.785/0.785/0.785; action F1 0.885/0.887/0.887/0.887). Performance saturates at 10000 features, and raising the cap beyond that changes nothing, simply because the corpus only contains about 10000 distinct unigrams and

bigrams in the first place. Even 5000 features capture 99.7% of the top performance. The classification signal is concentrated in a compact set of domain-specific terms, and the 20000-feature default is more than sufficient.

4.12 From the Benchmark to Complete Sessions

All results so far come from the AI-labelled dataset, in which trade actions are heavily overrepresented (Section 3.4). The final experiments therefore ask the question that actually matters for deployment: what does the best model do when it has to read a real session from start to finish? Five complete livestream sessions were replayed line by line, 18709 transcript lines covering 15.1 hours of streaming, and none of these lines were seen during training by the model replayed here. Logistic regression was trained once on the cleaned dataset minus these five sessions and then asked to classify every single line, with prompts built by the same machinery that produced the training data.

The replay answers two questions. First, does the model still find the actions it should find? Largely yes: of the 23 labelled actions in these sessions, 18 (78%) received an action prediction, 15 of them with exactly the right label, broadly in line with the action recall measured on the benchmark.

The second question is the decisive one: what else does the model fire on? Far too much. Over the unlabelled lines it flagged an action 3395 times, roughly 225 per hour, while the existing labels contain only 23 actions across all 15.1 hours. Why did the benchmark hide this? A base-rate effect. Whenever the target is rare, even a competent detector throws mostly false alarms; the same logic makes a reliable medical screening test return mostly false positives for a rare disease. On the benchmark about three lines in five are actions. In a complete session, the labelled actions make up fewer than one line in a hundred. Because the stream labels are known to be incomplete, that second figure is only an indication of the imbalance, not an exact action rate.

Of those 3395 fires, how many were actually right? To estimate this, a random sample of 100 fires (20 per session) was reviewed against the surrounding transcript. The review worked the same way the dataset itself was built: an AI model (Claude) read several lines of context around every fire and judged it, and the author then checked and confirmed the verdicts; every verdict is recorded with its reason in the evaluation files. The result: 3 of the 100 sampled fires were real trade actions that the dataset had simply never labelled, 5 were borderline, and 92 were clear false alarms. The model's standalone precision on a full stream is therefore roughly 3%, and at most 8% if every borderline case is counted in its favour, against 0.863 on the benchmark. The false alarms read like the error analysis of Section 4.10

at scale: market analysis that contains trade words, reviews of past trades, conditional plans, and advice to viewers.

Could a stricter decision threshold fix this? No. Only 3 of the 3395 unlabelled fires exceed the production system’s confidence gate of 0.74, which sounds promising until one checks the other side: only 1 of the 18 correctly detected actions exceeds that gate as well. The model is unsure about everything, false alarms and real actions alike, so any threshold strict enough to silence the noise also silences the signal. Run on its own, the classifier proves unusable on a real stream. The next measurement shows how the production system escapes that fate.

Two details of the replay setup deserve mention. First, every prompt includes the trader’s current position (long, short, or flat). In the live system that value comes from the broker, but the replay has no broker, so it rebuilds the position from the only ground truth available: the 23 labelled actions, where an entry opens a position and an exit closes it. Because there are only 23 of them across all five sessions, the rebuilt position reads “flat,” meaning no open position, almost all of the time. The training prompts looked much the same, but this skews the results in one way: when there is no open position, the management actions (trim, exit, move stop) do not fit, so when the model gets a line wrong it tends to guess an entry. Second, to check that the replay really showed the model its usual input, the prompts it rebuilt were compared with the prompts stored for these sessions during training. Most match character for character, so the model saw the input format it was trained on.

4.13 The Full Pipeline on the Same Sessions

Those figures all assume the classifier acting on its own, which the production system never lets it do. Production hands the primary decision to the rule engine, the bank of several hundred hand-written text patterns from Section 3.1, and casts the classifier in a purely assisting role: its confidence scores veto shaky rule matches and rescue actions the rules overlooked. For a final comparison, the same five sessions ran again through this actual interpretation pipeline under three setups: the rule engine by itself, the rule engine paired with the deployed ModernBERT classifier, and the rule engine paired with the benchmark winner, TF-IDF logistic regression, in the classifier slot. Throughout the replay the position state advanced from the pipeline’s own decisions, exactly as it would live, and a fired intent is credited with catching a labelled action when it falls within two lines of that label. Table 11 places the result beside the standalone numbers.

Table 11: The production interpretation pipeline on the five held-out sessions (15.1 hours), compared with the standalone classifier of Section 4.12. Fires are action intents on lines that carry no action label.

Configuration	Labelled actions detected	Exact label	Fires per hour
Standalone classifier (Section 4.12)	18 / 23	15	~225
Rule engine alone	5 / 23	5	5.4
Rule engine + ModernBERT (production)	7 / 23	7	6.2
Rule engine + TF-IDF logistic regr.	7 / 23	7	6.3

Three things stand out.

First, the pipeline fires far less often: about six times per hour instead of 225, a thirty-five-fold drop. And those fires are no longer mostly noise. Every fire from all three setups (99 distinct transcript lines) was reviewed against the surrounding transcript, using the same AI-plus-author check as the standalone sample. Among the 93 fires of the production setup, 15 turned out to be real trade actions that simply carry no label in the dataset (live calls such as “I’m out of this now” or “stops into break even”), 8 were borderline (usually the trader reporting a trade he had just made), and 70 were false alarms. So about one fire in six lands on a real action, against roughly one in thirty for the standalone model, and the false alarms drop from around 210 per hour to about 4.6 per hour.

Second, this caution has a price. Of the 23 labelled actions, the pipeline catches only 7, well short of the standalone model’s 18, because the rule engine turns down exactly the vague phrasings (“Okay. Out. Oh, let it squeeze out.”) that the statistical model would catch. For those seven detected labelled actions, the predicted label is correct in all seven cases. The system is built to miss a trade rather than invent one, and the numbers show that choice clearly.

Third, the classifier earns its place in this supporting role, but which classifier it is barely matters. Either one lifts detection from 5 to 7 of 23, at the cost of about one extra fire per hour. The deployed ModernBERT and the benchmark-winning TF-IDF model behave almost the same, right down to the share of fires that land on a real action. Once the rules make the main decision, the gap between the two model families almost disappears.

What does this mean for practical use? As a fully automatic trade copier, the pipeline is not good enough yet: it would still trigger around four to five unjustified trades per hour while missing two thirds of the labelled actions. As an alerting assistant, where a human sees each proposed trade and confirms or rejects it, the numbers are workable: the alert rate is low enough

to follow live, about every sixth alert points at a genuine action the dataset never knew about, and the detections it does make are labelled exactly right. The honest verdict is that the layered design turns an unusable classifier into a usable assistant, but not into an autonomous system.

5 Discussion

5.1 Why the Classical Models Win Here

The most prominent finding of this work is that the TF-IDF-based linear models outperform the frozen transformer-head models on the aggregate metrics. Given how thoroughly transformers dominate recent NLP benchmarks, this deserves a short explanation, and the evidence has already appeared along the way.

Three strands of the explanation reinforce each other. The TF-IDF features are learned entirely from the training data, so they capture the domain’s vocabulary automatically, while the frozen encoders, trained on general-purpose text, never had a reason to distinguish trading phrases such as “peeling some off” (a `TRIM`) from “taking this off” (`TRIM` or `EXIT_ALL`, depending on context). Trading language is also unusually repetitive: a small repertoire of fixed phrases announces most actions, and bigram features give each of these phrases its own dedicated dimension, which is exactly the sparse keyword signal the feature-importance analysis surfaced.

The comparison is also asymmetric by design: the classical models adapt every parameter to the task, while the transformers may only train their tiny heads, so the part of them that can actually specialize in trading language has roughly a tenth as many trainable parameters as a linear TF-IDF model. The t-SNE plots provide compatible but only illustrative evidence: in their two-dimensional projections, the seven classes remain intermixed. This result is consistent with Kowsari et al.’s survey observation that deep learning is unlikely to outperform other approaches when only a small text sample is available [15].

An important caveat applies. This study evaluates transformers exclusively in the feature-extraction setting (frozen encoder, trained head). The results show that *frozen pretrained embeddings* do not beat TF-IDF on this task; they do not show that transformers in general are inferior. A fully fine-tuned transformer, or one adapted with parameter-efficient methods such as LoRA, might close or reverse the gap. The comparison here is between fully task-adapted classical models and minimally task-adapted transformers. That is not a level playing field for the transformers, but it is the comparison

this small-data, limited-compute study set out to make.

5.2 The Precision-Recall Trade-off

For action detection, the five models fall at different points along a clear precision-recall spectrum. At the precision end sit the SVM and the balanced MLP, with action precision of 0.900 and 0.898; ModernBERT leans the other way, toward recall, at 0.917 action recall. Logistic regression lands in the middle. Its action precision of 0.863 and action recall of 0.914 yield the strongest action F1 of all.

This trade-off carries direct practical consequences, because the two error types cost different things. A false positive means executing a trade the trader never made; a false negative means missing a trade he did make. A deployment that prioritizes safety would favour a precision-leaning model such as the SVM, accepting that some signals are missed. One that prioritizes completeness would favour a high-recall model, that is, the transformers, accepting more false alarms that must then be caught by downstream safeguards. The balanced profile of logistic regression makes it the most versatile default.

5.3 Which Actions Are Hard, and Why

Across all five models the per-label results tell one consistent story. Management actions (`MOVE_STOP`, `EXIT_ALL`, `TRIM`) are easiest, landing at F1 around 0.70 to 0.85, for the reason Sections 4.3 and 4.6 established: traders announce them with explicit, recurring keywords. The `NO_ACTION` class falls in between (F1 roughly 0.72 to 0.83) and varies from model to model, because the line between no action and a subtly phrased action is genuinely blurry, the more so given the labels' incomplete action coverage. Entries (`ENTER_LONG`, `ENTER_SHORT`) are consistently hard (F1 roughly 0.48 to 0.64): entry language is varied and often implicit, and the model must also pin down the direction, which can hide in an earlier sentence. `MOVE_TO_BREAKEVEN` remains the unstable outlier, for the reason already established: there is almost no data for it.

The hierarchy also points to where improvement is cheapest. Entry classification has the most to gain from extra training examples that vary the phrasing. `MOVE_TO_BREAKEVEN` is different: the most pragmatic fix there is structural, not statistical. It could simply be folded into the closely related `MOVE_STOP` class, since moving a stop to breakeven is just a special case of moving a stop.

5.4 Implications for Production Deployment

Logistic regression won the benchmark, yet the production system keeps ModernBERT in the classifier slot, a decision that predates the benchmark. So should production simply adopt the stronger model? In isolation the answer is yes: logistic regression classifies somewhat more accurately and costs no more to run. The full-stream experiments, however, made the question far less important. Neither family produced a standalone classifier capable of carrying the live system, because on realistic input the real signals lie buried under false alarms, and no confidence threshold separates the two. The design that follows from this is forced rather than chosen. The rule engine fires rarely and precisely and owns the primary decision; the classifier only assists it, vetoing shaky rule matches and recovering clear misses. Inside that role, both models behaved almost identically in the pipeline replay, so replacing the deployed one would accomplish little. The choice instead turns on practical concerns, among them memory use and the ease of shipping a trained model, and any swap of a component would still demand an end-to-end evaluation that includes live execution. What truly moves the system forward is not the pick between these two classifiers but the training data itself (Section 5.7).

Latency warrants separate attention, since for trading it counts almost as heavily as accuracy. A signal that arrives seconds late executes at a worse price, and the production risk engine discards signals that have grown too old. The dominant cost inside the pipeline is the path from speech to finished text. Before anything else the system must wait out the trader’s utterance, and segments can run to roughly four seconds; transcription then contributes a few hundred milliseconds more, its queue held deliberately short so that a rapid speaker cannot accumulate a backlog of stale audio. Classification is trivial by comparison, a matter of milliseconds up to a few tens of milliseconds, whether the classical models or the transformer head does the work. The model chosen thus has almost no bearing on end-to-end latency. Speech-to-text governs it, and that is where any future latency effort should begin.

5.5 Ethical, Legal, and Operational Risk

Beyond classification accuracy, the application context brings its own risks. In a trade-copying system, a false positive is more than a labelling error: in a system connected to execution infrastructure, it could create an unintended market order. Any production deployment should therefore retain human override controls, confidence thresholds, stale-signal checks, position-size limits, and audit logs. The evaluation in this thesis measures text classification

only and should not be interpreted as evidence that fully automated execution is safe.

The data source also requires care. Livestream transcripts are public-facing media, but automated collection and reuse may still be constrained by platform terms, copyright, or streamer-specific expectations. This thesis uses archived transcripts for research; an operational system should verify that its collection, storage, and use of transcript data are permitted. Finally, detected trade signals should not be presented as financial advice. The system classifies what a speaker appears to say; it does not evaluate whether the trade is suitable, lawful, or economically sound.

5.6 Limitations

Several limitations of this work should be acknowledged.

Benchmark composition, and the scope of the full-stream test. The primary results (Sections 4.1 to 4.11) were measured on the curated benchmark corpus, in which trade actions are heavily overrepresented, and must not be read as deployment accuracy. The full-stream evaluation (Section 4.12) was added precisely to measure that gap, and it found a severe one: standalone precision of only a few percent on complete sessions. That test has limits of its own, however. It covers five sessions with only 23 labelled actions, so its recall estimate is coarse; the position state during the replay was reconstructed from the sparse labels and was therefore flat most of the time; and the precision estimate rests on a 100-fire sample, reviewed by an AI model and confirmed by the author, rather than on a fully labelled stream. The pipeline replay (Section 4.13) shares these limits, although its fires were at least all individually reviewed with the same AI-plus-author process. The benchmark results remain what they were designed to be, a controlled comparison between model families; the full-stream results give the direction and rough size of the deployment gap, not a precise production error rate.

Dataset size. With 1442 cleaned examples (1377 after deduplication) drawn from 103 transcripts, the dataset is small. Results may not carry over to larger or differently distributed datasets. The relative ranking of classical against transformer models could also shift given substantially more training data, particularly once full fine-tuning becomes feasible.

Single domain. All transcripts come from a single trader and a single instrument (Nasdaq futures). Different traders, instruments, or trading styles may use different language that requires separate training.

Frozen encoders. The transformer models were evaluated exclusively with frozen encoder weights. Full fine-tuning, parameter-efficient fine-tuning

(such as LoRA), or continued pretraining on trading-domain text might substantially improve their performance but were not explored.

Imperfect labels. The labels were generated by LLMs and reviewed by the author, and the manual verification (Section 3.4) found them precise but incomplete: not every action in the streams is labelled, some NO_ACTION lines may contain weak or repeated actions, and two accepted labels were ambiguous even to a human reader. The labels also record only what the trader announced; without broker statements there is no proof that an announced trade was actually placed. Finally, the labelled lines were not sampled at random: lines with clear trade wording were more likely to be selected for labelling in the first place. Such selection makes the classes appear cleaner than the underlying speech really is. It can also make the measured error rates, above all the false-positive rate, look better than they would on unselected speech.

Each line judged in isolation. Every transcript segment is judged on its own, even though trading actions arrive in sequences: entry, then management, then exit. Were a model aware of this temporal structure, it could rule out impossible predictions and lean on context to resolve ambiguous lines.

5.7 Future Work

Several directions follow from the findings and limitations.

Training with realistic class balance. The cleaned benchmark contains 829 action and 548 NO_ACTION examples, about three actions for every two non-actions. Its 2.5-to-1 cap on non-actions is therefore never activated. Complete streams are far more uneven, and the full-stream test (Section 4.12) showed exactly where that mismatch leads. The most direct route to closing the measured precision gap is retraining on far more non-action lines that contain trade words, precisely the market-analysis chatter that triggers the false alarms.

Fully labelled full-stream evaluation. The full-stream test relied on the 23 labelled actions of five sessions and a sampled review of the model’s fires. Labelling several entire sessions exhaustively, every line, would turn the sampled precision estimate into an exact one, allow recall to be measured beyond the existing labels, and give the position-state replay a complete action sequence to work with.

Full fine-tuning with more data. Expanding the labelled set substantially across multiple traders would make it possible to test full-encoder fine-tuning with validation-based regularization, instead of keeping the encoder frozen. This would show whether task-specific adaptation can close

the observed gap without assuming a particular sample-size threshold in advance.

Parameter-efficient fine-tuning. Methods such as LoRA [10] adapt a pretrained model by training only a small number of additional parameters, a middle ground between frozen feature extraction and full fine-tuning that could be feasible even at the current dataset size.

Sequence-aware modelling. The classifier currently judges every segment on its own, even though trades unfold in order: an exit can only follow an entry, and a stop can only be moved while a position is open. A model that reads consecutive segments together, or that carries the running position state from one prediction into the next, could rule out impossible actions and recover entries that only become clear over two or three lines.

Domain-specific pretraining. Continued pretraining on a large corpus of transcripts from publicly accessible trading livestreams could adapt the embeddings to trading vocabulary and reduce the mismatch with general-purpose pretraining data, provided that collection and reuse are permitted.

End-to-end evaluation with execution. Section 4.13 replayed the interpretation pipeline offline. An end-to-end test that adds the optional LLM confirmation layer, live transcription noise, and the risk and execution stages, ideally judged against broker records, remains to be done.

6 Summary

This work addressed the question of which machine learning approach provides the most accurate trade-intent classification from noisy livestream transcripts, comparing classical TF-IDF-based methods with frozen transformer feature extractors.

Question. Given a short transcript segment from a day-trading livestream and the current trading context, which model family best predicts the trader’s intended action from a set of seven classes (no action, enter long, enter short, trim, exit all, move stop, move to breakeven)? Which action classes are most and least amenable to automated detection, and what systematic error patterns emerge?

Procedure. From 103 transcribed YouTube livestreams, 1466 lines were labelled with the help of three large language models and reviewed by hand; cleanup and deduplication left 1377 unique examples for the primary evaluation. A manual check of 203 action labels confirmed nearly all of them, but also showed that not every action in the streams is labelled, so the benchmark is action-enriched rather than a complete record. Five models were compared under identical conditions: logistic regression, a linear SVM, and an MLP

on TF-IDF features, and DistilBERT and ModernBERT as frozen encoders, kept exactly as pretrained, with small trained classification heads. Evaluation used five-fold cross-validation, split by transcript so that no model is ever tested on a session it trained on. A closing experiment replayed five complete held-out sessions, more than 18000 lines, through the best model.

Results. On the cleaned dataset, logistic regression achieved the best macro F1 (0.703) and action F1 (0.887), while the SVM achieved the best accuracy (0.795) and action precision (0.900). ModernBERT achieved the highest action recall (0.917) but lower precision and macro F1. The MLP improved markedly with balanced oversampling (macro F1 from 0.572 to 0.634) but still trailed both linear models. Data cleanup improved most metrics. The supporting analyses explain the gap: classification is driven by a small set of domain keywords that TF-IDF captures directly, the frozen embeddings show no visible grouping by label in a two-dimensional t-SNE projection, and the dominant error modes are indirectly phrased entries, exit-like commentary, and advice to viewers mistaken for action. A vocabulary ablation showed performance saturating at about 10000 features.

The full-stream experiment then produced the second central finding of this work: the benchmark numbers do not transfer to complete sessions. Reading five full held-out sessions, the best model still found 18 of 23 labelled actions, but fired about 225 times per hour, of which only a few percent were real actions, so on its own it would be unusable live. The cause is the rarity of real actions in natural speech, and the consequence is the layered production design in which hand-written rules decide and the classifier merely assists. A replay of that full pipeline over the same sessions confirmed the design. It fires only about six times per hour, and a review of every one of those fires showed that about one fire in six lands on a real trade action the dataset never labelled. The classifier layer adds detections the rules miss, and the choice of assisting model, ModernBERT or TF-IDF, barely changes the system’s behaviour (Sections 4.12 to 5.4).

These results indicate that for small, domain-specific text classification tasks with repetitive, keyword-driven language, classical machine learning methods on TF-IDF features remain highly competitive. The added complexity of pretrained transformer feature extraction does not translate into better performance when the encoder is frozen and the training set is small.

For practitioners facing similar tasks, that is, classification on small, specialised, and often noisy corpora, the practical lesson is to start with simple, well-understood baselines before investing in heavier architectures, and to evaluate them under conditions as close as possible to deployment. The dominance of pretrained transformers on standard benchmarks does not automatically transfer to every setting; when the encoder cannot be fine-tuned

and the classification signal concentrates in a small set of domain keywords, a TF-IDF model with a balanced linear classifier is a competitive, interpretable, and easy-to-deploy choice.

The source code of the system and of the complete benchmark pipeline is publicly available at https://github.com/AndiCodesss/bachelor_thesis, together with the labelled dataset and the 103 recorded transcripts it was built from, so the repository holds the code, the data, and the stored results behind every number reported in this thesis (Section 3.8.3).

References

- [1] Johan Bollen, Huina Mao, and Xiaojun Zeng. Twitter mood predicts the stock market. *Journal of Computational Science*, 2(1):1-8, 2011.
- [2] Cristian Buciluă, Rich Caruana, and Alexandru Niculescu-Mizil. Model compression. In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, page 535-541, 2006.
- [3] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321-357, 2002.
- [4] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273-297, 1995.
- [5] Tri Dao, Dan Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems*, 35, 2022.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, page 4171-4186, 2019.
- [7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [8] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [9] David W Hosmer, Stanley Lemeshow, and Rodney X Sturdivant. *Applied Logistic Regression*. John Wiley & Sons, 3rd edition, 2013.
- [10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [11] Mingqing Hu and Bing Liu. Mining and summarizing customer reviews. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, page 168-177. ACM, 2004.

- [12] Thorsten Joachims. Text categorization with support vector machines: Learning with many relevant features. In *European Conference on Machine Learning*, page 137-142. Springer, 1998.
- [13] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.
- [14] Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, volume 2, page 1137-1143, 1995.
- [15] Kamran Kowsari, Kiana Jafari Meimandi, Mojtaba Heidarysafa, Sanjana Mendu, Laura Barnes, and Donald Brown. Text classification algorithms: A survey. *Information*, 10(4):150, 2019.
- [16] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, 2019.
- [17] Tim Loughran and Bill McDonald. When is a liability not a liability? textual analysis, dictionaries, and 10-Ks. *The Journal of Finance*, 66(1):35-65, 2011.
- [18] Christopher D Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [19] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [20] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26, 2013.
- [21] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32:8024-8035, 2019.

- [22] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825-2830, 2011.
- [23] Jeffrey Pennington, Richard Socher, and Christopher D Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, page 1532-1543, 2014.
- [24] Bethany Percha. Modern clinical text mining: A guide and review. *Annual Review of Biomedical Data Science*, 4:165-187, 2021.
- [25] Matthew E Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, page 2227-2237, 2018.
- [26] Matthew E Peters, Sebastian Ruder, and Noah A Smith. To tune or not to tune? adapting pretrained representations to diverse tasks. In *Proceedings of the 4th Workshop on Representation Learning for NLP (RepL4NLP-2019)*, page 7-14, 2019.
- [27] John C Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In Alexander J Smola, Peter L Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large Margin Classifiers*, page 61-74. MIT Press, 2000.
- [28] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *PMLR*, page 28492-28518, 2023.
- [29] Alexander Ratner, Stephen H Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269-282, 2017.
- [30] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5): 513-523, 1988.

- [31] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [32] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427-437, 2009.
- [33] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [34] SYSTRAN. faster-whisper: Faster whisper transcription with CTranslate2. <https://github.com/SYSTRAN/faster-whisper>, 2023. GitHub repository, accessed 23 July 2026.
- [35] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579-2605, 2008.
- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [37] Mayur Wankhade, Annavarapu Chandra Sekhara Rao, and Chaitanya Kulkarni. A survey on sentiment analysis methods, applications, and challenges. *Artificial Intelligence Review*, 55(7):5731-5780, 2022.
- [38] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.
- [39] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, page 38-45, 2020.

- [40] Frank Z Xing, Erik Cambria, and Roy E Welsch. Natural language based financial forecasting: A survey. *Artificial Intelligence Review*, 50 (1):49-73, 2018.
- [41] Yi Yang, Mark Christopher Siy Uy, and Allen Huang. FinBERT: A pretrained language model for financial communications. *arXiv preprint arXiv:2006.08097*, 2020.